

Programmation WEB

Noircir les cercles comme montré : ○○●○

Connaissances générales

Plusieurs bonnes réponses possibles

Barème : Bonne réponse +1, mauvaise réponse ou réponse partielle -0,5, pas de réponse 0

Q. 1 Quel mot-clef HTTP permet de supprimer une ressource ?

- ☒ DELETE
- ☐ DROP
- ☐ POST
- ☐ PUT

Q. 2 Le langage Kotlin est un langage dérivé et compatible avec le langage ?

- ☒ Java
- ☐ C++
- ☐ Python
- ☐ JavaScript

Q. 3 Parmi les URL suivantes la/lesquelle(s) sont (est) incorrecte(s) ?

- ☒ `https :\toto.com\t/o/t/o`
- ☐ `http ://tata.fr`
- ☒ `httpm ://monSite.com/my/sub/page.html`
- ☐ `https ://fr.wikipedia.org`

Q. 4 Quel code HTTP signale une erreur du serveur ?

- ☐ 204
- ☐ 401
- ☒ 504
- ☐ 102

Q. 5 Soit `maListe`, une liste contenant les entiers de 1 à 10 inclus . Que fait la fonction `maListe.filter { it % 2 == 0 }` ?

- ☐ Elle renvoie la liste 1, 3, 5, 7, 9
- ☒ Elle renvoie la liste 2, 4, 6, 8, 10
- ☐ Elle renvoie la liste 2
- ☐ Elle renvoie la liste 2, 4, 8

Questions ouvertes :

Barème : Bonne réponse 1 point

Q. 6 À quoi correspondent respectivement les couches HTML, CSS et JavaScript ?

Q. 7 Que permet de faire un mapper dans le code d'un serveur ?

Un mapper permet de réaliser la conversion entre le type récupéré en base de données, l'entité et celui à renvoyer par le controller, le DTO. Il permet notamment d'ignorer les propriétés inutiles pour le DTO ou de composer les propriétés venant de plusieurs entités.

Q. 8 Que signifie la lettre I du principe SOLID ?

Le I signifie Interface Segregation Principle, il précise que l'on doit définir des interfaces répondant à un besoin précis, et ne pas avoir une interface générique couvrant plusieurs besoins.

Q. 9 Quel est le principal défaut de l'égalité abstraite de JavaScript ?

L'égalité abstraite est une égalité relative, elle ne permet notamment pas de faire la différence entre 0, null, undefined et la chaîne vide.

Q. 10 Quelle est la différence entre du code synchrone et du code asynchrone, et dans quel cas va-t-on utiliser l'un plutôt que l'autre ?

Le code synchrone exécute chacune des instructions de manière séquentielle : une instruction ne démarre pas tant que la suivante n'est pas terminée.

Le code asynchrone permet d'enchaîner les instructions sans nécessaire attendre les résultats.
Le code asynchrone s'utilise particulièrement pour tout ce qui va toucher aux interactions avec le serveur, rien ne garantissant quand la réponse sera reçue.

Le code synchrone s'utilise dans la plupart des cas pour les instructions à réaliser lors d'une action en particulier.

- Q. 11** Soit une table de données contenant la liste des élèves d'une école, avec comme informations la date de naissance, le numéro de sécurité sociale, le nom, la classe.

Écrire une requête (SQL ou JPA) permettant de récupérer tous les élèves qui sont en CE1

SQL :

```
SELECT * FROM ELEVES WHERE CLASSE='CE1'
```

JPA :

```
SELECT ELEVES FROM ELEVES WHERE CLASSE='CE1'
```

- Q. 12** Expliquer le rôle d'un serveur web

Un serveur web permet de surveiller l'intégrité des données, de les sauvegarder dans une base de données, de contrôler les actions réalisées par les utilisateurs et de traiter les différentes requêtes envoyées par le client.

- Q. 13** Expliquer le rôle d'un client web

Requête les données au serveur, s'assure de leur mise en forme et de l'affichage.

Il est responsable des interactions qui peuvent être réalisées par l'utilisateur.

- Q. 14** Dans une application comportant les packages suivants : controllers, services, mappers, entities, repositories, dans quel package dois-je placer les fonctions correspondant respectivement aux requêtes HTTP et aux requêtes SQL ?

Les fonctions correspondant aux requêtes HTTP doivent être placées dans le package controllers.

Les fonctions correspondant aux requêtes SQL doivent être placées dans le package repositories.

- Q. 15** Décrire succinctement le principe du DOM.

Le principe du DOM est de construire un arbre représentatif de l'application.

Il va ainsi représenter le code HTML en présentant des blocs successifs qui vont servir avec le CSS à faire le rendu de la page.

Questions de pratique

Barème : Bonne réponse : 3 points.

Remarque : Les questions suivantes nécessitent beaucoup de texte à écrire, et partent du principe que vous travaillez sur une application déjà fonctionnelle ~~contrairement à celle fournie lors du TP4.~~

- Q. 16** Écrire un exemple de code HTML, contenant uniquement les définitions des différents blocs (html, style, script) à leur place.

```
<html>
  <header>
    <style></style>
  </header>
  <body>
    <script></script>
  </body>
</html>
```

- Q. 17** Écrire un controller permettant de réaliser les opérations de récupération, création, modification et suppression des données. Ce controller s'appellera `ComputerController` et utilisera un service appelé `ComputerService` qui mettra à disposition les méthodes `getAllComputer`, `getComputerById`, `createComputer`, `deleteComputer`, `updateComputer`.

```
@RestController
class ComputerController(
    private val computerService: ComputerService
) {
    @GetMapping
    fun getAllComputer(): ResponseEntity<List<ComputerDTO>> {
        return ResponseEntity.ok().body(computerService.getAllComputer());
    }

    @GetMapping("/{id}")
    fun getComputerById(@PathVariable id: Long): ResponseEntity<ComputerDTO> {
        return ResponseEntity.ok().body(computerService.getComputerById(id));
    }

    @PostMapping
    fun createComputer(@RequestBody newComputer): ResponseEntity<ComputerDTO> {
        return ResponseEntity.ok().body(computerService.createComputer())
    }

    @DeleteMapping("/{id}")
    fun deleteComputer(@PathVariable id: Long): ResponseEntity.HeadersBuilder<
        computerService.deleteComputer(id);
        return ReponseEntity.noContent()
    }

    @PathMapping("/{id}")
    fun updateComputer(@PathVariable id: Long): ResponseEntity<ComputerDTO> {
        return ResponseEntity.ok().body(computerService.updateComputer(id));
    }
}
```

- Q. 18** Écrire le repository qui fera les requêtes SQL correspondantes, permettant de récupérer la liste de tous les ordinateurs, de mettre à jour un ordinateur en changeant la valeur contenue dans sa colonne `NAME`.

```
@Repository
interface ComputerRepository: JpaRepository<Computer, Long> {
    @Query("SELECT * FROM COMPUTER")
    fun findAll(): List<Computer>

    @Query("UPDATE COMPUTER c SET c.NAME= ?1 WHERE c.ID = ?2")
    fun updateComputerName(newName: String, id: Long): Computer
}
```

- Q. 19** En JavaScript, il existe une fonction qui permet de capturer les touches du clavier : `addEventListener("keydown", (event) =>{ })`, où `event` possède un attribut `code` qui contient le détail de la clé appuyée. Sachant que la clé de la touche espace est égale à `"space"`, écrire un code JavaScript permettant de déterminer la longueur d'un mot tapé.

```
let length = 0;
```

```
function lengthCounter(event) {
  if (e.code === 'Space') {
    myHtmlBlock.text = length;
    length = 0;
  } else {
    length++;
  }
}
```

```
myInput.addEventListener("keydown", lengthCounter);
```

- Q. 20** Soit `myObservable`, un `Observable` émettant successivement les valeurs 2, "etre", 4, "soir", [1, 2, 3]. écrire la suite d'opérations qui permet d'extraire les valeurs numériques, puis de les multiplier par 2. Pour s'assurer qu'une valeur "maValeur" en JavaScript est bien un nombre, on peut utiliser la suite d'opération : `const numberValeur = Number(maValeur); Number.isNaN(numberValeur);` La deuxième opération renvoyant `false` si `maValeur` est un nombre et `true` sinon.

```
myObservable.pipe(
  filter((element) => !Number.isNaN(Number(element))),
  map(element => element * 2))
.subscribe();
```

Bonus :

Barème : Les questions qui suivent sont optionnelles, les points pour une bonne réponse sont ajoutés à la note finale.

- Q. 21** Citer une bibliothèque de tests unitaires, ou de mutation (1 point)

- Karma
- Jest
- Jasmine
- JUnit
- Striker Mutator
- Pitest
- vitest

- Q. 22** Expliquer comment écrire un test permettant de valider unitairement une fonction (2 points)

L'idée d'un test permettant de valider unitairement une fonction est d'écrire une méthode qui appelle avec une valeur simple pour tous les paramètres d'entrée, puis de réaliser la même exécution avec les paramètres aux limites, par exemple, ceux qui provoquent des exceptions pour le code du serveur, ou avec les valeurs `null` ou `undefined`.

L'idée est de mettre en entrée suffisamment de cas pour couvrir l'ensemble des branches conditionnelles du code, sans pour autant en avoir deux qui couvrent le même cas.